Olfaction Communication System

Andrew Bernas, Minoo Kim, Samantha Liu, Winston Liu, Shivani Sahni, Kaishuu Shinozaki-Conefrey, Dhruv Shrivastava, Anahita Srinivasan, Alexandra Chin, Erik Luna, Alan Yu & Pulkit Tandon

Abstract

The sense of smell, also known as olfaction, is very important to all animals and their everyday lives. It allows us to perceive danger, such as the smell of fire or burning, as well as help animals in the wild detect food. Being able to recognize a smell and reproduce it in any given area can invoke a plethora of applications. We designed a frugal and open-source prototype of an end-to-end device that can identify the odorant, and replicate that odor across any length of distance. This allows users to effectively communicate a wide range of smells with each other. The prototype system consists of three sub-blocks: 'e-nose', communication system and synthesis. Electronic nose, or 'enose' allows computers to detect smells. By obtaining the data from various volatile organic compound (VOC) sensors through a sensor chamber, e-nose enables identification and synthesis of the given smells we are targeting. This data is communicated between the e-nose and an odor synthesizer through the usage of bluetooth and wifi modules, along with Google's online database (Firebase) that will identify and transmit smell information. Lastly by connecting an Arduino microcontroller to a ceramic humidifier disc, we can vaporize any given liquid based odor into thin air, and therefore synthesize odors detected by the electronic nose.

Background

Olfaction, the process and action of smelling, has been replicated through the use of various sensors and modules, resulting in the creation of an "e-nose". In order to bridge the communication between the "e-nose" and a synthesizing device that replicates the smell, detection and communication programs are necessary to create a working pipeline. Through the use of tools such as Python scripts, Firebase, and Android app development, establishing a connection between the e-nose and synthesizer becomes more streamlined. Creating a communication pipeline will allow for the communication and synthesis of smells from large distances.

Many attempts have been made already to create algorithms to accurately identify different smells. For instance, researchers at Brown University[1] created an electronic nose named

TruffleBot that uses VOC sensors as well as pressure and temperature sensors. The device was accurately able to differentiate between drinks such as vodka, beer, wine, and lime juice. Through in-depth research on existing work and brainstorming other applications, we found that odor synthesizing devices can help in the medical field, real estate field, education field, and more. Inhaling various essential oils is known to have certain benefits, such as improving memory, relieving stress, curing mild infections, and increasing attention span and alertness. Studies[2] have shown that inhaling certain scents can have psychophysiological effects on humans. Electroencephalographs (EEGs) are used to measure responses in the central nervous system as well as the physiological state of the brain at any given time. EEGs are generally used to detect brain disorders such as dementia or brain death, but some have reported that EEGs can detect spontaneous changes in the brain when a scent is inhaled. Various EEG studies conducted relating to olfactory stimulation have suggested that inhaling certain scents can alter cognition, mood, and social behavior. Smells can also benefit the performance of people's learning and memorizing. A study found that emitting the scent of peppermint can arouse attention, allowing us to stay focused and concentrate on the task at hand[3]. Smell synthesizers, such as Aromajoin's Aroma Shooter and MIT Media Lab's Project Essence[4] and BioEssence[5], have been utilized to create odors in accordance to the user's emotional state, influencing mood and cognitive performance. Despite the countless benefits of inhaling essential oils, inhaling too much, even when diluted, can cause various health risks, especially for children. For instance, inhaling too much peppermint essential oil can cause respiratory problems for infants[6]. With a limit in available sensor technology, the range of detectable odors reduces, which restricts usable smells in synthesizing. These limitations stem from the number of commercially available sensors that focus highly on detecting odorless smells.

Methods and Materials

We used an array of specialized VOC sensors, like the MQ-3 (Alcohol and Ethane), the MQ-136 (Hydrogen Sulfide), the MQ-137 (Ammonia), and the MQ-138 (Benzene, Toluene, Alcohol, Propane, Formaldehyde, Hydrogen). These sensors allowed us to detect unique features of a smell, which we then used to create a signature used to identify the smell again in the future. Other methods can also be employed to expand the feature space, increase consistency, and minimize error. These methods include (but are not limited to) passing the smell through multiple unique filters in parallel and using fans to reset the sensor chamber. To administer the smell, the essential oils and other scents were placed on a tissue and put next to the sensors. For the "e-nose" itself, the sensor chamber we constructed allowed us to test and run trials in a favorable environment. The fans attached to the chamber were hooked up to a second Arduino and allowed for a constant flushing of smells, which made the data more accurate since the smells weren't lingering and interfering with the actual sensor readings. When we tested without

the chamber, the readings were all over the place and unusable because there was no difference within the data that we could utilize to differentiate the variety of smells, demonstrating the necessity of having the chamber. In addition, we established a procedure to repeat every time we collected data: first, we would let the sensors warm up for 60 seconds, then we allowed the smell to saturate the sensor chamber for 30 seconds, before reading data for another 60 seconds. Finally, we gave the sensor chamber 300 seconds to reset to air.

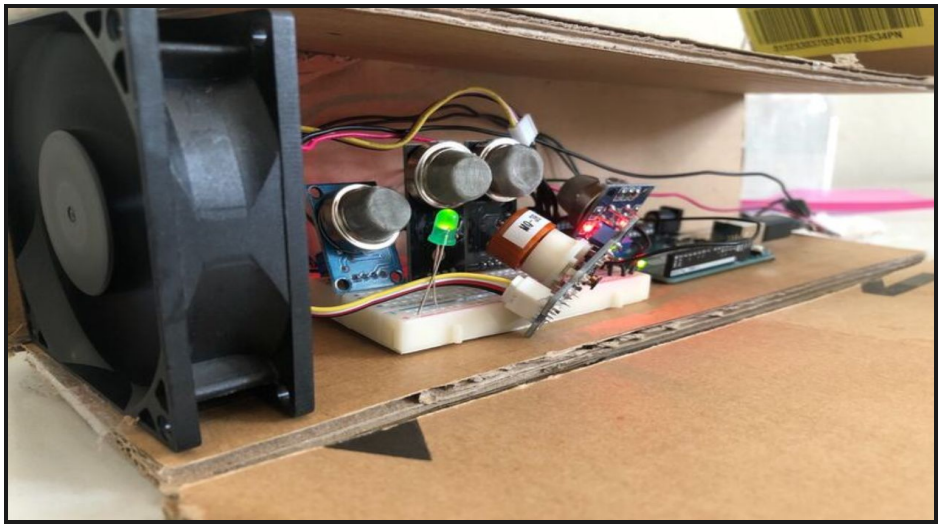


Figure 1: Enose set up (Open)

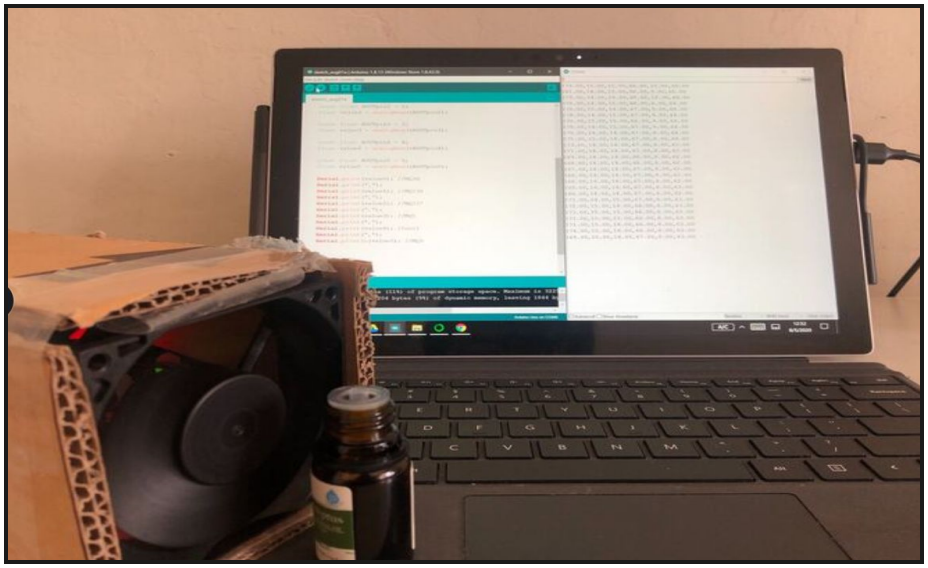


Figure 2: Enose on and detecting a smell

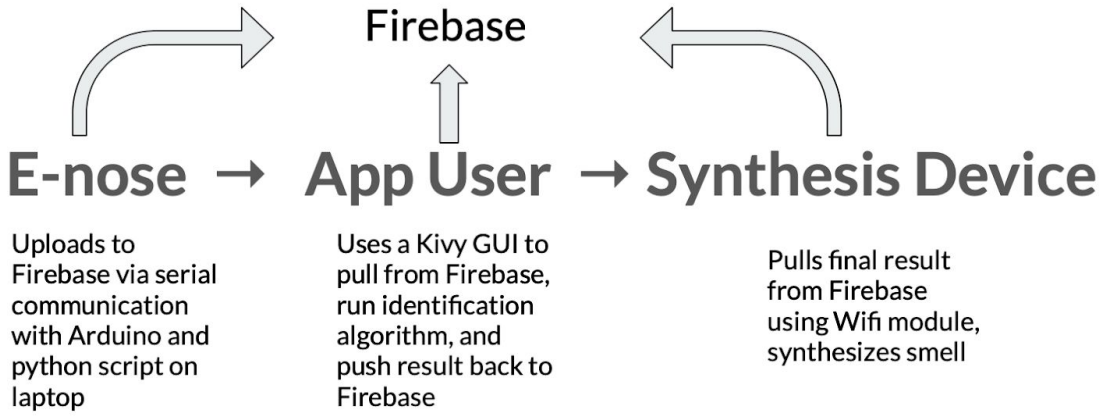


Figure 3: Diagram of communication pipeline

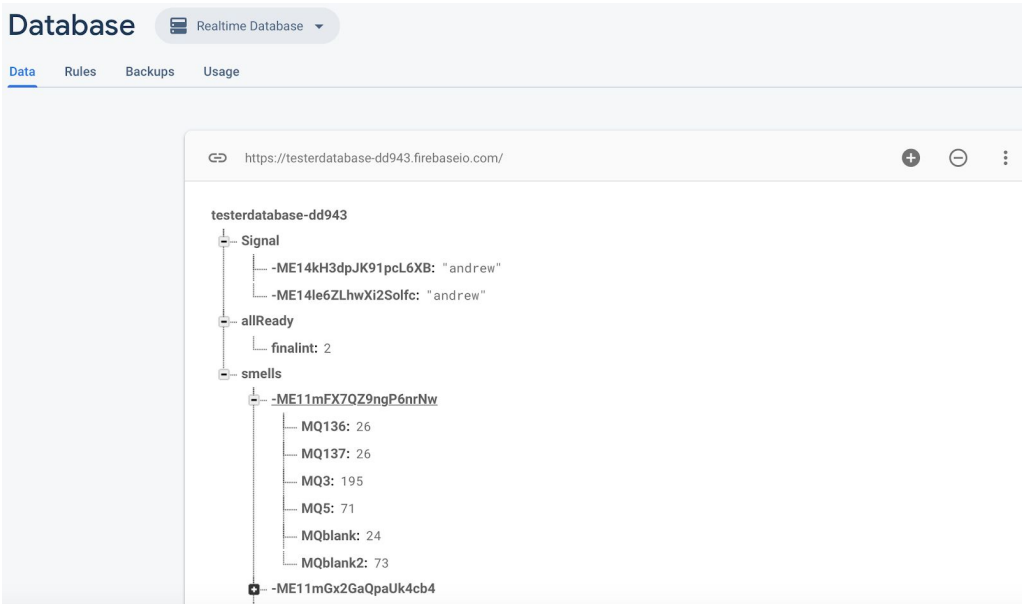


Figure 4: Firebase Database that stores the raw data from the e-nose, readiness signal from the synthesizing device, and final identified smell.

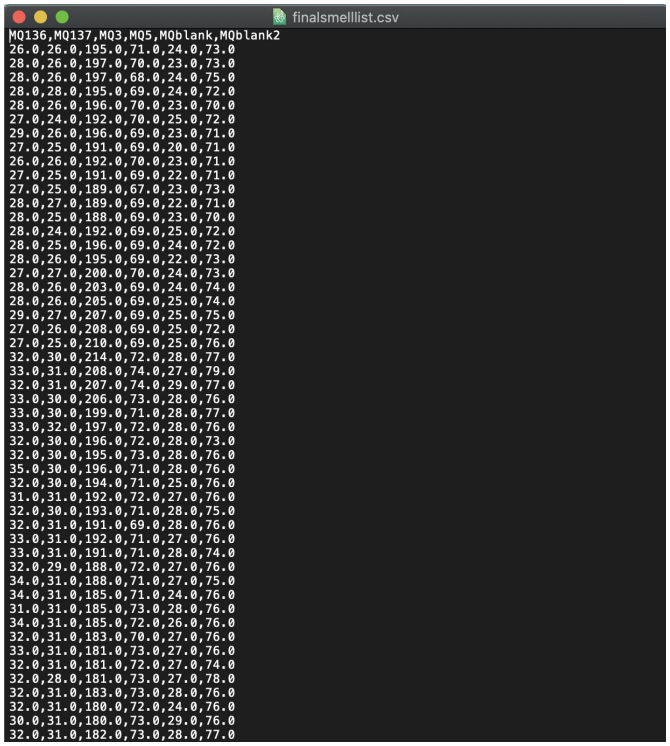


Figure 5: Example raw data from e-Nose displayed in a CSV file

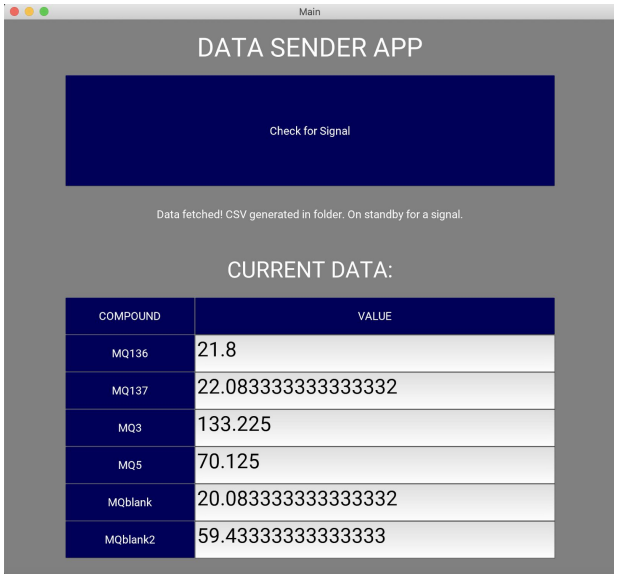


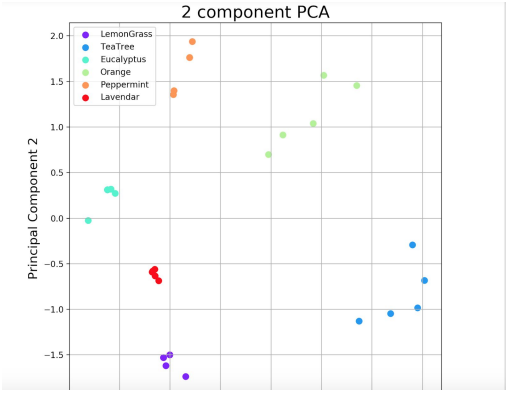
Figure 6: Kivy Graphical User Interface with averaged data values from each of the sensors of the sensor array.

One of the most important components in the design of our communication system is Google's Firebase database. We have created one database through this service that contains all of the data points needed for the project. As shown in Figure 4, we have a folder entitled "smells" that

contains the raw data the e-nose sensors emit, organized by time; this folder contains sixty individual JSON structures, each of which contains the sensor readings taken at a particular second. There is also a folder entitled “Signal” to which the synthesis device pushes a string once it is ready, and several different folders, each with the name of a different ready signal, that hold the final smell identity as an integer.

In order to visualize each step of the communication pipeline, the Olfactory Communication System [16] uses Kivy, an open source cross-platform Python framework that allows for the creation of a Graphical User Interface (GUI) using Python rather than a different programming language. The Kivy framework was chosen deliberately to allow for the excellent data processing tools needed for the smell identification algorithm in our project that are only available in the Python programming language. The GUI (Figure 6) facilitated the communication pipeline by fetching raw data from the database, displaying average data from each sensor in the sensor array, generating a comma separated value file with the raw data (Figure 5), running the identification algorithm, and sending the final result of the identification process back to the Firebase database. The user is able to easily tell which step of the pipeline is being executed, and can choose when they would like to send or receive data, as well as who to send it to.

The basic communication pipeline (Figure 3) entails sending raw data from the Arduino E-Nose to a Firebase database in the cloud, and then having a desktop application that can pull data from the database, run an identification algorithm, and deliver the final result to another Arduino that can reproduce the smell through the database. The first step of building the identification algorithm involved collecting 5 trials of training data from each smell, with each trial containing a minute’s worth of reading. Principal component analysis was then performed on this data to assist in visualizing the six sensors’ data as a 2 dimensional graph in which clusters, indicating the several trials of smells of the same type, could be recognized and labeled. K-means[7] was used to help the computer recognize these data clusters and provide tools to predict the cluster which future data would belong to.



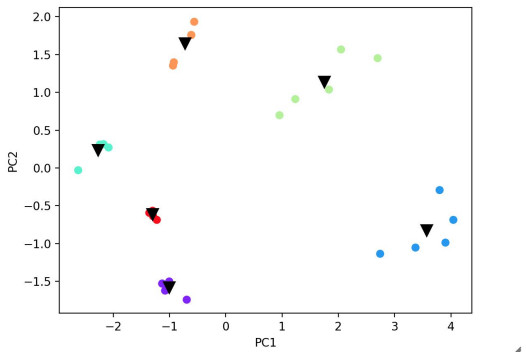


Figure 7: The initial graphed set of training data, which appeared to have well-defined clusters
Some issues arose from this method of scaling and performing PCA on the data; because the nature of PCA uses the entire set of data to determine how best to reduce the number of dimensions to only two, future data recorded in isolation would need to be operated on in the same way. This meant that principal component vectors and scaling factors had to be collected for each set of training data, meaning whenever the training data was changed, the algorithm had to be "recalibrated" so that new data could be classified into the clusters.

```
    #x is the averaged data from newly collected smell samples.
    means = np.array([166.37231638,81.09830508,71.52429379])
    var = np.array([2793.13435762,13.00845351,12.14645089])


    components = np.array([[-0.63914269,0.52454393,-0.56245025],
[-0.06739724,-0.76670794,-0.63844854]])

    x = ((x-means)/var**0.5)
    comp1 = np.dot(x,components[0])

    comp2 = np.dot(x,components[1])

    #Comp1 and comp2 signify the two dimensions used after the PCA is performed.
```

Figure 8: Newly collected data had to be processed through a scaling formula, and then the dot product of this scaled data and vectors from the original PCA compressed the newly collected data to two principal components

However, it was discovered that faulty methods had produced these clean clusters; each trial had been performed in succession, meaning that often lingering traces of the first trial would cause the second trial to appear to have similar data, and so on. The consequences of this meant that future smell data, recorded in isolation, would not place anywhere near the smell cluster it should have. Another round of training data was collected, this time with each trial given time to allow the previous smell to dissipate completely before the next trial began.

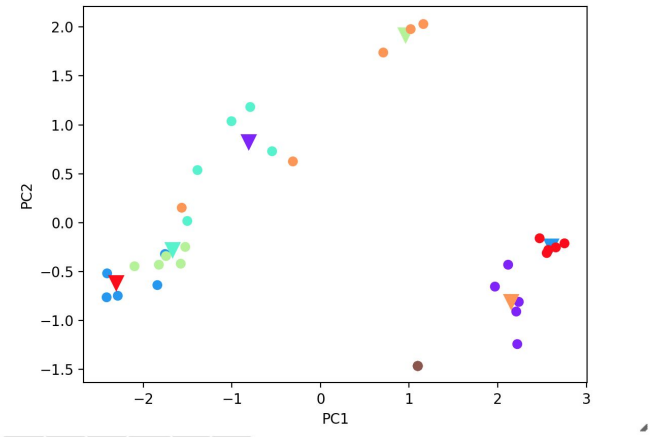


Figure 9: Slightly noisier but still distinguishable data from the second round of training data

Further issues arose out of this new training data, however. Future smell data still did not match up to the previously established clusters, and by examining the raw data from several sensors, it was found that certain sensors varied little between smell, but could vary significantly simply depending on the time each smell was recorded as the sensor responded to environmental influences. Because the PCA observed those sensors having very little deviation in value, we decided to choose only one sensor, the alcohol sensor, to identify smell data with, and future data was classified by simply using nearest neighbor identification on the means of each set of smell data. The results of the single-sensor graph is shown in Figure 10.

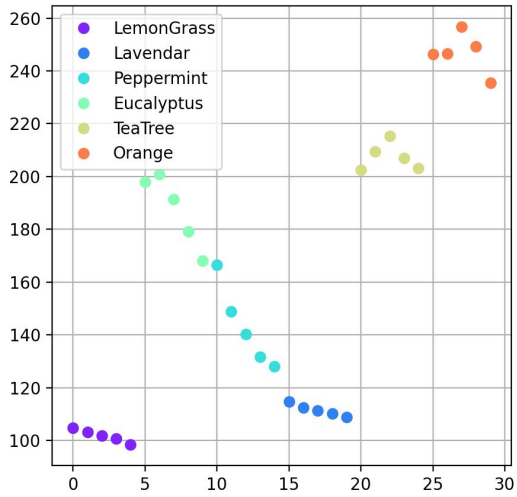


Figure 10: Training data organized by nothing more than a single sensor's data

This new algorithm provided a more robust way to categorize the specific set of smells from very limited readings and was used in the final demo to correctly classify the smell of eucalyptus.

Once the smell has been identified, the application waits for a signal from the synthesis device that indicates readiness to receive the smell, and sends the final result back to the database once the signal has been received. The synthesizing device pulls the final result and produces the corresponding smell.

In sending raw data from the e-nose to the Firebase database, a combination of serial communication and a Python script was used for communication. The Arduino microcontroller of the e-nose sends its collected data to a laptop via a USB cable, which a Python script then accesses and sends to Firebase. A Firebase library available on both Github and PyPi facilitated the pushing of e-nose data to the Firebase database from the Python script.

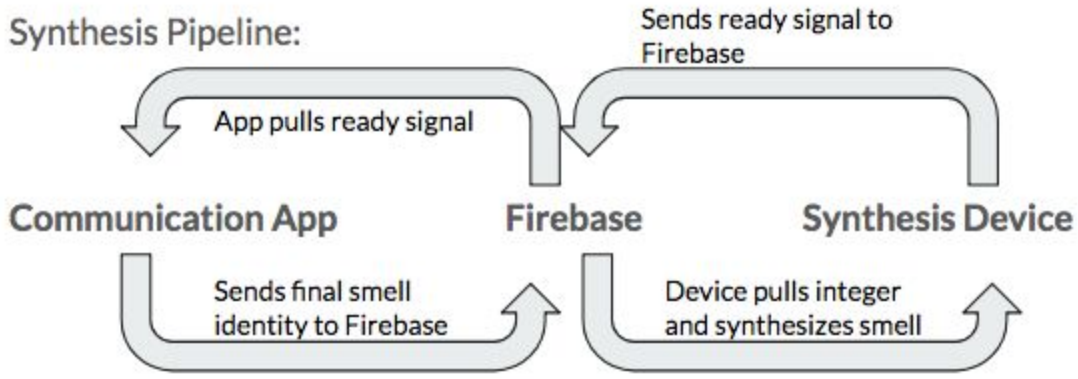


Figure 11: Final synthesis pipeline showing synthesis device's indirect communication with Kivy app using Firebase

In receiving data from Firebase and sending a signal to indicate readiness, the ESP-01 wifi module (part of the ESP8266 line of wifi modules) on the Arduino microcontroller of the synthesis device was used in order to facilitate communication. The wifi module connects to the database and sends a string to the database when the user is ready to receive a smell, and later pulls the final result from the database after the app has pushed it to the database. A library entitled FirebaseArduino, available on Github, was used to allow the microcontroller to communicate with the database.

The wifi module was significantly more effective and easier to use than serial communication via a USB cable. The wifi module was capable of both sending and receiving data without any major changes to the code, whereas the serial cable had some more issues with two-way communication. When two devices are connected serially, only one device can access the serial port at a time to either write or read data. One-way communication (as seen with the e-nose) is relatively simple, but allowing both devices to communicate back and forth requires either a handshake protocol or set time intervals for each device to connect and disconnect from the serial port.

In terms of designing the synthesis device, we brainstormed 4 possible solutions: 1. Spray Bottle 2. Pressurized Spray Canister 3. Solid-state Smells 4. Mist Maker. Using a spray bottle is a very crude design that would consist of a small spray bottle and a servo motor to squeeze the trigger and release the liquid-based smell (essential oils). However, spray bottles are not very robust as their pump mechanism wears out over time, and we would also need a servo motor capable of pulling the trigger fast enough and with enough torque, which would pose a major challenge as many servos are built for one of the traits but not both. The pressurized spray canister would consist of a pressure pump, airtight container, and a spray nozzle. The pump will essentially

build enough pressure inside the container and when released, the liquid-based smell would be pushed out the spray nozzle and emitted into the air. This solution is over-complicated and will pose a major issue in the building the pressure chamber due to our lack of tools. This also defeats our purpose of using readily available parts.Using solid state smells, like Aromajoin[8], would allow a very clean and efficient way of swapping out different smells and prevent any mess caused by liquid based odors. We decided not to pursue this idea because of various limitations in tools to create these. Solid scents require heat[9] to be effectively emitted, which was another obstacle that steered us away from this solution. Finally, our group decided on the mist maker solution[10]. This design incorporates mist makers that are capable of vaporizing any liquid based odor into the air. We would control the mist maker by connecting a MOSFET to it. This would allow us to send a signal to it via a microcontroller and seamlessly switch it on and off. Due to this relatively simple design, we choose to pursue this idea. Our mist makers contain an encapsulated piezoelectric transducer which is capable of vibrating at a frequency of 100KHz when using alternating current (AC) voltage. This vibration creates an ultrasonic sound which is undetectable by the human ear, but is rapid enough to break liquid water into vapor particles.

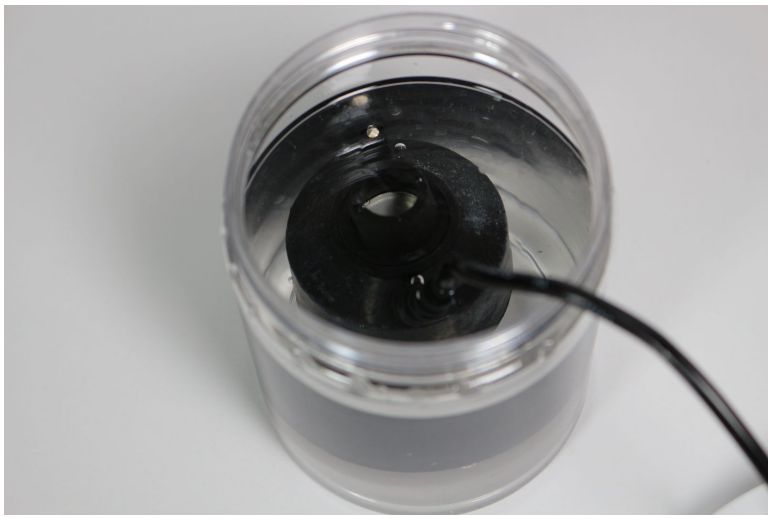


Figure 12: Mist maker inside our synthesis container.

After building our first working prototype we discovered that the vapor couldn't escape the enclosure, so we implemented a fan to help blow the vapor out of the enclosure. By aiming a fan into the synthesis container, pressure builds allowing the scent to emit through an output hole. We also attached a straw to the opening of the enclosure to help direct the vapor in one direction.

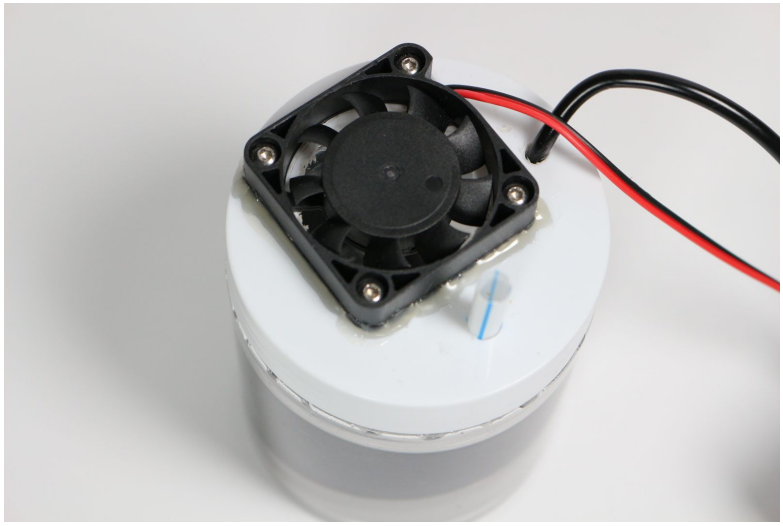


Figure 13: Fan and straw mounted onto the lid of the synthesis container

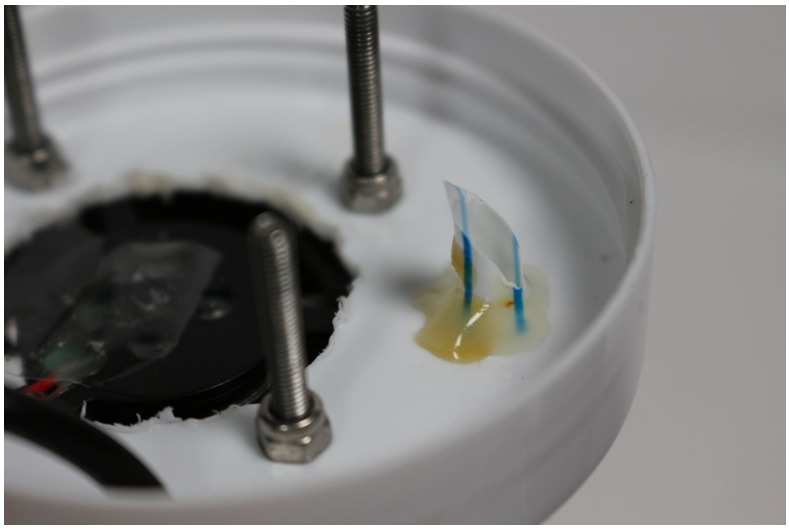


Figure 14: Note how we cut the straw diagonally to increase the surface area of the entrance and prevent clogging.

We used an ESP8266-01 WiFi Module as a microcontroller to switch our MOSFETs on and off. This finger nail size board has 4 GPIO pins and WiFi capabilities allowing us to interface with the communication app. Each GPIO pin corresponds to an individual synthesis container. For example, GPIO pin 0 is connected to two MOSFETs which are hooked up to a fan and mist maker which are powered via a 24V power supply. The ESP8266-01 is rated at 3.3V but we and many before us found success with powering it with 5V. Thus, we used a step down voltage regulator rated at 5v to power our microcontroller from the 24V power supply.

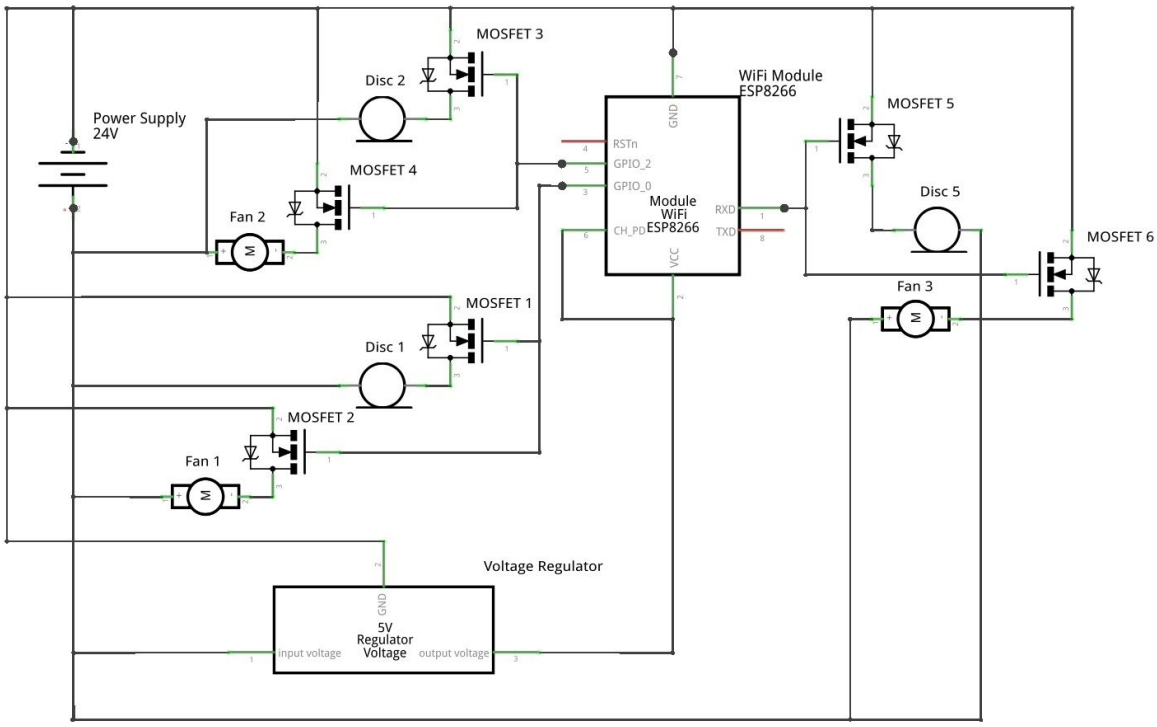


Figure 15: Schematic of our completed circuit.

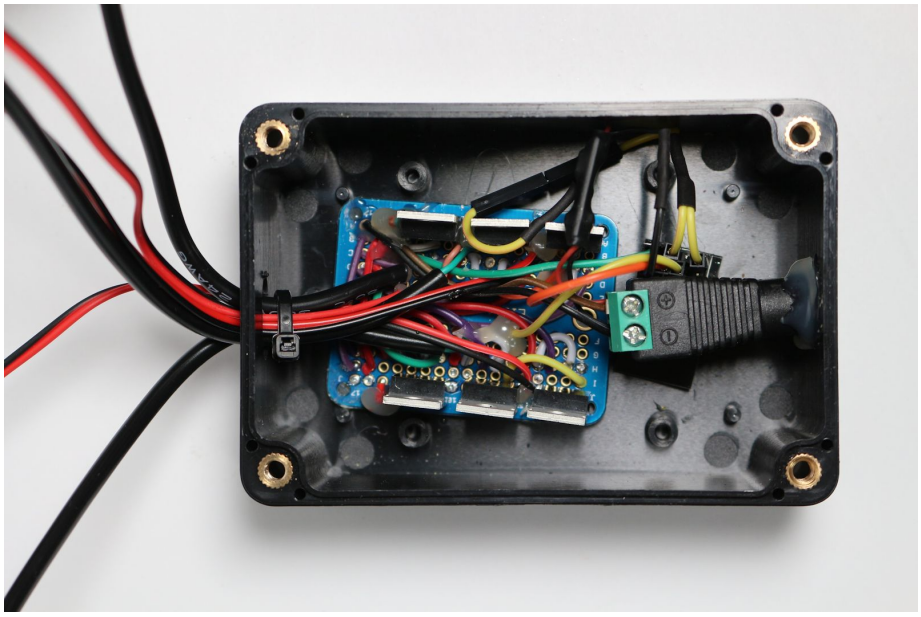


Figure 16: Electrical components inside the black enclosure.

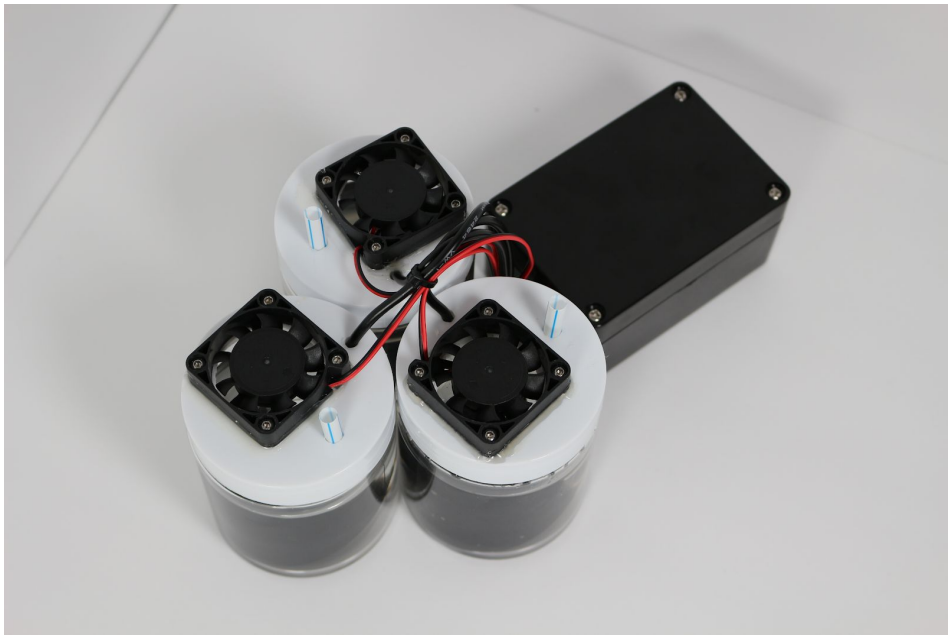


Figure 17: The completed synthesis device.

The odors we selected to detect and emit consisted of a variety of essential oils, hand sanitizer, and an odor neutralizer to get rid of any lingering smells when switching between scents. Hand sanitizer was chosen to suit the alcohol sensor, as its alcohol content is very high. The essential oils included eucalyptus, peppermint, orange, tea tree, lavender, and lemongrass as many of these smells are correlated to benefiting effects. We chose these because though the sensors were not targeted for these scents, we wanted scents that were both safe to reproduce and pleasing to smell.

Results

We were able to enact our final pipeline and communicate smells such as that of eucalyptus from the e-nose to the emitter, with the user on the synthesis side correctly identifying the smell that the e-nose user transmitted without being explicitly told what it should have been. Our smell algorithm was able to classify smells with reasonable accuracy using only one sensor and a simple nearest neighbor algorithm.

By connecting to Wifi using the ESP8266-01 module, the synthesis device was able to emit scent with user input. For example, by sending a ready signal to communication using the module, the data could be sent back in the form of an integer and easily reproduced by a corresponding synthesis machine. However, only one of four Wifi modules were able to work, limiting the

number of available smells to be sent and produced. The emitted smells were concentrated and easily detectable as the output allowed for direct flow in one area, avoiding sporadic release.

Conclusion

Our results and final pipeline did demonstrate that end-to-end communication of smell data is possible and can be frugal. However, our pipeline required many manual steps; for example, the process of sending the e-nose's sensor data to Firebase took two steps that each had to be triggered manually and carefully watched for completion. There were also many limitations in recognizable and reproducible smells. The physical components of our olfactory system did not have the fidelity to sense differences in many of the smells we chose, with our conclusion being we needed either more discernible smells based off of their chemical information, or higher fidelity sensors of a greater range. Because of this lack of fidelity, we found that simply using the data of one sensor, the alcohol sensor, along with a nearest neighbor algorithm based on the means of the training data, actually provided the most accurate results. With a more optimal sensor setup, the cluster algorithm could have proven to be more useful.

There are many inaccuracies with the synthesis machine and improvements are necessary mainly including the simultaneous dispersion of smells and reliability of devices. Because three machines were created, only one could be set off at a time which decreases flexibility. Further, many materials were unusable due to fragility and the software occasionally produced inconsistent results such as the Arduino IDE not uploading code to the ESP8266-01 module or not having the code actually compile. This left us with only one working synthesis device out of the four we built, because three of the four WiFi modules short-circuited during testing.

Because the design includes vast amounts of wiring and hardware, the system lacks portability. Also, as the devices are connected to each other via one breadboard, all three machines need to stay within close range and cannot separate. The odor neutralizer slightly mitigates this problem of all smells being produced in one area, but still requires further improvement.

Through the synthesis device, scents are easily emitted for the user's benefit. This product effectively reproduces scent over long distances, and thus, is an adequate form of communication.

Future Directions

We hope to continue to improve our e-nose and its potential for countless beneficial applications. We plan to enhance the consistency of the testing environment in order to ensure the accuracy of the data. This will be done by constructing a better-suited sensor chamber as well as developing a more thorough method of administering the smells to achieve a uniform concentration. These would minimize the fluctuations in the sensor readings which would allow for easier differentiation of the smells and increase the efficiency of synthesizing the actual smells. Aside from the environment itself, we also hope to obtain and implement better sensors that would allow us to have better sensor readings for other types of smells and not just for odorless and harmful gases. Having optimal sensors would in turn allow us to have even more smells to test and utilize in the olfaction pipeline. Being able to detect the smell of fresh, baked cookies and synthesizing that in, for example, a retirement home, would allow the residents to feel comfortable and relaxed. With more smells, we would be able to expand our horizons and the applications of this standardized and frugal olfactory system.

One potential improvement that we would like to make to our communication protocol is making our user interface more user friendly, along with packaging it for a potential mobile app. We would like to use Kivy or possibly another user interface tool to make a more aesthetically pleasing interface that can be easily packaged into a mobile application. Additionally, we would like to continue automating the communication process, with less manual user input. Other aspects we hope to work on in the future include fixing the key system, adding multiple e-noses to the system, and connecting to multiple synthesis machines simultaneously. Overall, we plan to enhance our pipeline so that it is more seamless and automated.

A potential addition to our synthesis device is water level sensors. These water level sensors would detect when there is too little water for the mist makers to effectively produce vapor and will send a signal to the communication app to notify the user, adding a little more automation to the device. A challenge we faced during the build process of the synthesis device was the lack of GPIO pins. The ESP8266-01 WiFi Module only had a total of 4 data pins. This limits our number of synthesis containers by 4 and also prevents us from adding other devices like water sensors. Using an Arduino Uno WiFI or NodeMCU would prevent this issue from occurring and we would sacrifice a small form factor with the benefit of more pins for more devices. We would also like to implement a more advanced lid system. Our current solution works but the blowing fans can get a little annoying. To solve this issue we would either use lower powered fans or invest in higher quality fans to reduce the noise output. We would also like to improve our lid design to allow the user to easily exchange smells. The current issue is that because the mist maker cord is connected to the lid, removing the lid can get a little difficult when the wire gets in the way. This can be solved by mounting the cable on the side of the container rather than the lid.

In the end, we just want the user to have a seamless experience exchanging smells from the device.

Also, we want to increase the number of smells our synthesis device is able to produce. To do so, we would need either more small containers that each contain a scent, or a larger container that can emit several different scents, which would allow for a larger variety of scents to be produced. Another method of increasing the number of scents our device can produce is layering smells. We would have two containers emit their scents at the same time, allowing the two scents to mix in the air to create a new, unique scent. Additional methods of expanding the smells we can produce include solid scents, food, objects with scents, or other non-liquid alternatives, all of which would require further research.

Acknowledgements

We would like to acknowledge Professor Tsachy Weissman, Professor of Electrical Engineering, for his guidance and assistance in creating the Stanford Compression Forum Internship. We would also like to thank Suzanne Sims for her help in ordering parts and delivering materials to each of our homes and Cindy Nguyen for running the STEM to SHTEM internship and providing us with the opportunity to explore new topics, perform research, and learn from various experts in the STEM field.

Contributions

- E-nose: Winston Liu, Dhruv Shrivastava, Alan Yu
- Communication and Processing: Alexandra Chin, Shivani Sahni, Kai Shinozaki-Conefrey, Anahaita Srinivasan
- Synthesis: Andrew Bernas, Minoo Kim, Samantha Liu, Erik Luna

References

[1] Webster, J., Shakya, P., Kennedy, E., Caplan, M., Rose, C. and Rosenstein, J. (2018). *TruffleBot: Low-Cost Multi-Parametric Machine Olfaction - IEEE Conference Publication*. [online] Ieeexplore.ieee.org. Available at: https://ieeexplore.ieee.org/document/8584767 [Accessed 3 Jul. 2019].

[2] https://www.ncbi.nlm.nih.gov/pmc/articles/PMC5198031/

[3] Shannon Barker, Palema Grayhem, Jerrod Koon, Jessica Perkins, Allison Whalen and Bryan Raudenbush. 2003. Improved Performance on Clerical Tasks Associated with Administration of Peppermint Odor. Perceptual and Motor Skills (2003), 1007–1010.
[4] https://dam-prod.media.mit.edu/x/2017/04/25/essence.pdf
[5]https://dam-prod.media.mit.edu/x/2018/05/24/bioessence-EMBC_JSPZpIS.pdf
[6]https://healthywa.wa.gov.au/Articles/A_E/Essential-oils#:~:text=What%20are%20the%20dangers%20of,if%20small%20amounts%20are%20ingested.
[7] https://scikit-learn.org/stable/modules/generated/sklearn.cluster.KMeans.html
[8]https://aromajoin.com/products/aroma-cartridge
[9] https://fultonandroark.com/blogs/news/how-to-apply-solid-cologne
[10] https://www.youtube.com/watch?v=8elsCIGOCWE

[9] https://kivy.org/doc/stable/api-index.html
[10] https://github.com/FirebaseExtended/firebase-arduino
[11] https://pypi.org/project/firebase/
[13] https://drive.google.com/file/d/1MrXekyv4wBD0UsDD8JyErcEvnbGFjRZt/view
[14] https://drive.google.com/file/d/1Cd3HBeCMpo2Tw9Xf5LbvlMvDDcW_PgJq/view
[15] Macías, M., Agudo, E., Manso, A., García Orellana, C. and Gallardo Caballero, R. (2013).*A Compact and Low Cost Electronic Nose for Aroma Detection*. [online] Available at: https://www.ncbi.nlm.nih.gov/pmc/articles/PMC3690013/ [Accessed 3 Jul. 2019].
[16] https://github.com/acguarana6/OlfactionCommunicationSHTEM